ECE 276A Project 3 Report

Qihao Qian

UCSD ECE Department
A69030355
q2qian@ucsd.edu

I. Introduction

Visual-inertial simultaneous localization and mapping (SLAM) is a crucial technique for autonomous navigation in unknown environments. This project implements a visual-inertial SLAM system using an Extended Kalman Filter (EKF) to estimate the robot's pose and map static landmarks by fusing high-frequency IMU data with stereo camera measurements. The integration of these sensors not only enhances accuracy but also compensates for individual limitations, making the system robust against sensor noise and challenging conditions. This report details our methodology, experiments, and evaluation, showcasing the effectiveness of our approach in real-world scenarios.

II. Problem Formulation

A. EKF-Based IMU Localization on SE(3)

We want to estimate the IMU pose $\mathbf{T}_k \in SE(3)$ (i.e., $\mathbf{R}_k \in SO(3)$ and $\mathbf{p}_k \in \mathbb{R}^3$) over time using an EKF prediction step.

State and Inputs

Define the state as

$$\mathbf{T}_k = (\mathbf{R}_k, \mathbf{p}_k) \in SE(3),$$

where $\mathbf{R}_k \in SO(3)$ is the rotation and $\mathbf{p}_k \in \mathbb{R}^3$ is the translation. The IMU measurements provide the linear velocity $\mathbf{v}_k \in \mathbb{R}^3$ and angular velocity $\boldsymbol{\omega}_k \in \mathbb{R}^3$.

Process Model

We describe the continuous kinematics in discrete form as

$$\mathbf{R}_{k+1} = \mathbf{R}_k \, \exp\big(\boldsymbol{\omega}_k \, \Delta t\big), \quad \mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k \, \Delta t.$$

In $SE(3)$ notation, this can be written as

$$\mathbf{T}_{k+1} = \mathbf{T}_k \, \exp\big(\boldsymbol{\xi}_k \, \Delta t\big),$$

where $\boldsymbol{\xi}_k$ (the "twist") combines the angular and linear velocities, and the process noise is $\mathbf{w}_k \sim \mathcal{N}\big(\mathbf{0}, \mathbf{Q}\big)$.

EKF Prediction

- **State Prediction:**

$$\hat{\mathbf{x}}_{k+1}^{-} = f\big(\hat{\mathbf{x}}_k^{+}, \, \mathbf{u}_k\big),$$

- **Covariance Prediction:**

$$\mathbf{P}_{k+1}^{-} = \mathbf{F}_k \, \mathbf{P}_k^{+} \, \mathbf{F}_k^{\top} + \mathbf{G}_k \, \mathbf{Q}_k \, \mathbf{G}_k^{\top},$$

where $\mathbf{u}_k = \big(\mathbf{v}_k, \, \boldsymbol{\omega}_k\big)$ is the control input, and $\mathbf{F}_k$, $\mathbf{G}_k$ are the Jacobians of f with respect to the state and the noise, respectively.

Goal

Our goal is to minimize the pose estimation error

$$\|\mathbf{T}_k - \mathbf{T}_k^{*}\|$$

under Gaussian noise, thereby providing the best linear unbiased estimate of $\mathbf{T}_k$ and its covariance $\mathbf{P}_k$.

B. Landmark Mapping via EKF Update

Assuming that the IMU trajectory (pose) is already known or sufficiently accurate, we aim to estimate the 3D positions of static landmarks. The Extended Kalman Filter (EKF) uses visual measurements to update the landmark state estimates.

State and Inputs

Stack all landmark positions into a single state vector:

$$\boldsymbol{\mu} = \begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ x_2 \\ y_2 \\ z_2 \\ \vdots \\ x_N \\ y_N \\ z_N \end{bmatrix}.$$

The known camera pose $\mathbf{T}_k$ acts as the system input at each step. Visual observations $\mathbf{z}_k$ (typically pixel coordinates) correspond to feature locations associated with each landmark. Since the camera pose is assumed to be known (or accurately measured), there is no prediction step for $\boldsymbol{\mu}$; only a measurement update is performed.

Measurement Model

The camera projects each landmark $\boldsymbol{\mu}_i$ into the image plane based on the current camera pose $\mathbf{T}_k$:

$$\mathbf{z}_k = h(\boldsymbol{\mu}_i, \mathbf{T}_k) + \mathbf{v}_k,$$

where $\mathbf{v}_k$ represents the measurement noise (e.g., pixel noise).

EKF Update

For each landmark's measurement, the measurement function

h is linearized around the current estimate $\hat{\boldsymbol{\mu}}_j$. The measurement Jacobian for landmark j at time k is given by

$$\mathbf{H}_{k,j} = \left.\frac{\partial h}{\partial \boldsymbol{\mu}_j}\right|_{\boldsymbol{\mu}_j=\hat{\boldsymbol{\mu}}_j}.$$

Collecting the contributions from all landmarks, let $\mathbf{H}$ denote the overall Jacobian and $\mathbf{R}_k$ the measurement covariance. The EKF update proceeds as follows:

- **Kalman Gain:**

$$\mathbf{K}_k = \mathbf{P}^- \mathbf{H}^\top \Big(\mathbf{H}\,\mathbf{P}^-\,\mathbf{H}^\top + \mathbf{R}_k\Big)^{-1}.$$

- **State (Mean) Update:**

$$\hat{\boldsymbol{\mu}}^+ = \hat{\boldsymbol{\mu}}^- + \mathbf{K}_k\,\Big(\mathbf{z}_k - h\big(\hat{\boldsymbol{\mu}}^-\big)\Big).$$

- **Covariance Update:**

$$\mathbf{P}^+ = \big(\mathbf{I} - \mathbf{K}_k\,\mathbf{H}\big)\,\mathbf{P}^-.$$

Goals and Cost Function

Our aim is to accurately estimate each landmark's 3D position $\boldsymbol{\mu}_i$. A common cost function is the mean-squared reprojection error across all landmarks. By repeatedly applying the EKF update with new visual observations, we can maintain an up-to-date estimate of $\boldsymbol{\mu}$ and its covariance $\mathbf{P}$.

C. Visual-inertial SLAM

Overview

We fuse IMU measurements (angular velocity $\boldsymbol{\omega}_t$ and linear acceleration $\mathbf{a}_t$) with stereo camera observations (2D feature points from each camera). The quantities to be estimated are:

- The IMU pose $\mathbf{T}_t \in SE(3)$ (including orientation and position).
- The 3D positions $\mathbf{L}_j \in \mathbb{R}^3$ of multiple landmarks.

The EKF alternates between a *prediction step* (using IMU data) and an *update step* (using stereo measurements).

State Definition

At time t, the EKF state is defined as

$$\mathbf{X}_t = \big[\mathbf{T}_t,\ \mathbf{L}_1,\ \mathbf{L}_2,\ \ldots,\ \mathbf{L}_n\big],$$

where $\mathbf{T}_t$ represents the current IMU pose, and each $\mathbf{L}_j$ denotes a landmark position in the global frame.

Process Model (Prediction)

Using the IMU inputs $(\boldsymbol{\omega}_t, \mathbf{a}_t)$, we predict the new pose:

$$\mathbf{T}_{t+1} = f\big(\mathbf{T}_t,\ \boldsymbol{\omega}_t,\ \mathbf{a}_t\big).$$

This yields the predicted state $\mathbf{X}_{t+1}^-$. The state covariance is then propagated using the EKF Jacobians and the process noise covariance $\mathbf{Q}_t$.

Measurement Model (Update)

When the stereo camera observes landmarks, each landmark $\mathbf{L}_j$ is associated with its 2D projection by

$$\mathbf{z}_{t,j} = h\big(\mathbf{T}_t,\ \mathbf{L}_j\big).$$

The measurement update employs the measurement Jacobian and incorporates the measurement noise covariance $\mathbf{R}_t$ to update the state and its covariance.

Goals

1) Accurately track the IMU pose $\mathbf{T}_t$.
2) Minimize the overall estimation error by appropriately tuning the noise covariances $\mathbf{Q}_t$ and $\mathbf{R}_t$.

III. Technical Approach

A. EKF-Based IMU Localization

1) **IMU Calibration (Preprocessing)**
Although not strictly required for the core algorithm to run, IMU calibration is an essential preliminary step. It ensures that the inertial measurements are consistent. Common calibrations address potential biases or scaling factors in the measured linear and angular velocities, and are typically performed offline.
2) **State Representation**
The pose is represented by a 4×4 homogeneous transformation matrix:

$$\mathbf{T} \;=\; \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}^\top & 1 \end{bmatrix},$$

where $\mathbf{R} \in SO(3)$ is the rotation matrix and $\mathbf{p} \in \mathbb{R}^3$ is the position vector.
3) **Exponential Maps for Rotation**
The exponential map is used to represent small rotation errors on $SO(3)$, enabling a concise and continuous representation of angular updates. In implementation, functions such as `hat_so3` construct the skew-symmetric matrix associated with a vector in $\mathbb{R}^3$, while `exp_so3` computes the matrix exponential for rotation updates.
4) **EKF Prediction Step**
   a) **Noise Representation**
   Let $\mathbf{w}_k$ be the process noise, assumed Gaussian with covariance $\mathbf{Q}$.
   b) **State Transition**
   The discrete-time pose update can be written as

$$\mathbf{T}_{k+1} \;=\; \mathbf{T}_k\,\exp\big(\boldsymbol{\xi}_k\,\Delta t\big),$$

   where $\boldsymbol{\xi}_k$ (the "twist") incorporates both rotational and translational components. In code, this often uses the skew-symmetric operator via `hat_so3` for the rotation part.
   c) **Covariance Propagation**
   The covariance is propagated according to

$$\mathbf{P}_{k+1}^- \;=\; \mathbf{F}_k\,\mathbf{P}_k^+\,\mathbf{F}_k^\top \;+\; \mathbf{G}_k\,\mathbf{Q}\,\mathbf{G}_k^\top,$$

   where $\mathbf{F}_k$ is the state-transition Jacobian, and $\mathbf{G}_k$ is the noise Jacobian.
5) **EKF Update Step**
   a) **Jacobian Matrices**
   The measurement or state-transition functions are linearized around the current estimate, yielding Jacobians $\mathbf{H}$ (for measurements) or $\mathbf{F}_k$, $\mathbf{G}_k$ (for prediction).

b) **Kalman Gain**
Given a measurement $\mathbf{z}$ and measurement Jacobian $\mathbf{H}$ with covariance $\mathbf{R}$, the Kalman gain is

$$\mathbf{K} \;=\; \mathbf{P}^{-}\,\mathbf{H}^{\top}\Big(\mathbf{H}\,\mathbf{P}^{-}\,\mathbf{H}^{\top}+\mathbf{R}\Big)^{-1}.$$

c) **State and Covariance Update**
The state (e.g., pose or landmark positions) and covariance are updated as

$$\hat{\mathbf{x}}^{+} \;=\; \hat{\mathbf{x}}^{-} + \mathbf{K}\,\big(\mathbf{z}-h(\hat{\mathbf{x}}^{-})\big), \quad \mathbf{P}^{+} \;=\; \big(\mathbf{I}-\mathbf{K}\,\mathbf{H}\big)\,\mathbf{P}^{-}.$$

6) **Covariance Re-initialization**
In certain scenarios (e.g., large changes in sensor noise characteristics or re-calibration events), the filter may reset or partially re-initialize the covariance to reflect newly introduced uncertainties in the state parameters.

7) **Iterative Application Over Time**
A function such as `imu_localization_ekf` applies the EKF prediction step iteratively across all timestamps:

$$\Delta t_i = t_i - t_{i-1}, \quad i = 1, 2, \ldots, N.$$

For each step, the updated pose $\mathbf{T}_i$ and covariance $\mathbf{P}_i$ are computed and stored, yielding $\{\mathbf{T}_0, \mathbf{T}_1, \ldots, \mathbf{T}_N\}$ and $\{\mathbf{P}_0, \mathbf{P}_1, \ldots, \mathbf{P}_N\}$.

8) **Summary of Implementation**
Key Functions:
- `ekf_prediction_step`: Executes one EKF prediction step (state and covariance propagation).
- `imu_localization_ekf`: Applies the entire EKF pipeline to process a sequence of IMU measurements over time.

This structured approach, featuring concise mathematical equations and modular function design, ensures that readers with no prior background can follow the process of IMU-based localization using an Extended Kalman Filter.

B. Landmark Mapping via EKF

The goal is to estimate 3D landmark positions by fusing stereo image measurements with IMU-based poses using an Extended Kalman Filter (EKF). The overall procedure can be summarized in the following steps:

1) **Key Landmark Selection**

At each time step:
- For each landmark j, count its valid observations.
- Select those landmarks whose number of observations exceeds a chosen threshold (e.g., $\#\text{obs} > \tau$).

2) **Sparse Covariance Initialization**

Define the state vector for all landmarks as:

$$\mathbf{M} \;=\; \big[x_1,\, y_1,\, z_1,\, x_2,\, y_2,\, z_2,\, \ldots\big]^{\top}.$$

The initial covariance $\mathbf{P}$ is block-diagonal, with each 3×3 block set to $\text{diag}(\sigma, \sigma, \sigma)$, reflecting that each landmark's initial uncertainty is independent of others.

3) **Landmark Measurement Model**

When the stereo camera observes the i-th landmark, the measurement is given by

$$\mathbf{z} \;=\; h\big(\mathbf{T}, \mathbf{L}_i\big) \;+\; \mathbf{v},$$

where $\mathbf{T}$ is the known camera pose, $\mathbf{L}_i$ is the landmark's position in world coordinates, and $\mathbf{v} \sim \mathcal{N}\big(\mathbf{0}, \mathbf{R}\big)$ is the measurement noise.

4) **EKF Update**

For each new measurement, we linearize around the prior estimate $\mathbf{M}^{-}$ and update as follows:

a) **Kalman Gain:**

$$\mathbf{K} \;=\; \mathbf{P}^{-}\,\mathbf{H}^{\top}\Big(\mathbf{H}\,\mathbf{P}^{-}\,\mathbf{H}^{\top}+\mathbf{R}\Big)^{-1}.$$

b) **State Update:**

$$\mathbf{M}^{+} \;=\; \mathbf{M}^{-} \;+\; \mathbf{K}\,\big(\mathbf{z} - h(\mathbf{M}^{-})\big).$$

c) **Covariance Update:**

$$\mathbf{P}^{+} \;=\; \big(\mathbf{I} - \mathbf{K}\,\mathbf{H}\big)\,\mathbf{P}^{-}.$$

Additional checks include:
- *Chi-Square Gating:* Accept the measurement only if $(\mathbf{z} - \mathbf{z}_{\text{pred}})^{\top}\mathbf{R}^{-1}(\mathbf{z} - \mathbf{z}_{\text{pred}}) < \text{threshold}$.
- *Outlier Rejection:* If the gating condition fails, discard the measurement.

5) **Re-Entrance Processing**

At each time frame t:
- Gather the available stereo measurements $\mathbf{z}$ and their corresponding landmark indices.
- Compute the prediction errors and Jacobians $\mathbf{H}$ for each observed landmark.
- Perform the EKF update steps as above.
- Estimate updated positions for out-of-frame observations, if needed.
- Re-apply the measurement model and update the global state vector $\mathbf{M}$ and its sparse covariance $\mathbf{P}$ over time.

This structured procedure, with concise examples and partitioned batch processing, provides a clear pathway for performing stereo landmark mapping using an EKF, even for readers without prior experience in 3D triangulation.

C. Technical Approach

Our system fuses inertial measurements with stereo visual observations using a joint Extended Kalman Filter (EKF). The pipeline consists of the following components:

1) **IMU Localization (EKF Prediction)**

a) **State Representation**
The IMU pose is represented by a 4×4 homogeneous transformation matrix:

$$\mathbf{T} \;=\; \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}^{\top} & 1 \end{bmatrix},$$

where $\mathbf{R} \in SO(3)$ is the rotation matrix and $\mathbf{p} \in \mathbb{R}^3$ is the translation vector.

b) **Motion Propagation**
Given angular velocity $\boldsymbol{\omega}$ and linear velocity $\mathbf{v}$ over a small time Δt, the updates are:

$$\mathbf{R}_{k+1} = \mathbf{R}_k \exp(\hat{\boldsymbol{\omega}}\,\Delta t), \quad \mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k\,\Delta t.$$

The state covariance is propagated with

$$\mathbf{P}_{k+1}^{-} = \mathbf{F}_k\,\mathbf{P}_k^{+}\,\mathbf{F}_k^{\top} + \mathbf{G}_k\,\mathbf{Q}\,\mathbf{G}_k^{\top},$$

where $\mathbf{Q}$ is the IMU process noise.

c) **Covariance Propagation**
The matrices $\mathbf{F}_k$ and $\mathbf{G}_k$ are the state-transition and process-noise Jacobians, respectively. They are computed at each time step to propagate the uncertainty in the pose estimate.

2) **Joint Visual Update (EKF Correction)**

a) **Landmark Measurement**
The state includes both the IMU pose $\mathbf{T}_k$ (and rotation/error state) and the 3D landmark positions $\mathbf{L}_i$. When each landmark is observed by both cameras, the measurement is modeled as

$$\mathbf{z}_i = h\big(\mathbf{T}_k,\, \mathbf{L}_i\big) + \mathbf{v},$$

where $\mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$ is the measurement noise.

b) **Joint Update Step**
We linearize the measurement Jacobian $\mathbf{H}$ around the current pose and landmark estimates, then apply the EKF correction. The Kalman gain is computed as:

$$\mathbf{K} = \mathbf{P}^{-}\,\mathbf{H}^{\top}\Big(\mathbf{H}\,\mathbf{P}^{-}\,\mathbf{H}^{\top} + \mathbf{R}\Big)^{-1},$$

and the state and covariance are updated accordingly. The IMU pose is updated on $SE(3)$ using the exponential map.

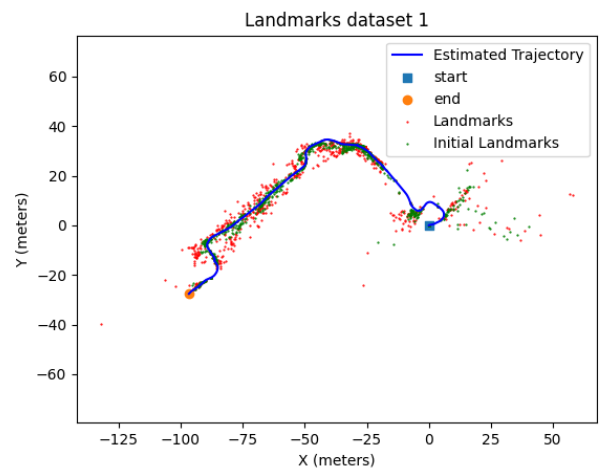
3) **Pipeline Summary**

- **IMU Calibration:** (Preprocessing step to remove sensor biases; assumed to be performed before using raw measurements.)
- **IMU Prediction:** Use the IMU EKF to propagate the pose with the above motion model.
- **Landmark Initialization:** Select landmarks with sufficient observations and compute their initial positions via multi-view triangulation.
- **Visual Measurement Processing:** For each frame, project the 3D landmarks to both cameras using intrinsic and extrinsic calibration, then compute the residuals and corresponding Jacobians.
- **Joint EKF Update:** Fuse the visual measurements with the inertial prediction to jointly update the IMU pose and the landmark estimates.

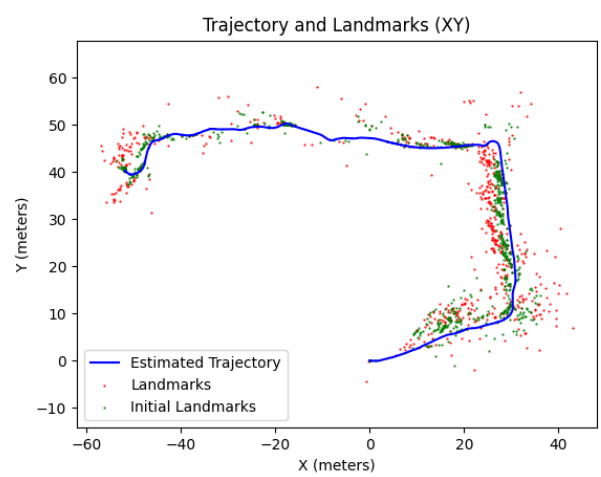
This modular approach—with clearly defined mathematical models and update equations—provides a complete roadmap for implementing visual-inertial SLAM, even for readers with no prior background.

IV. Result

The image shows the trajectory obtained from my path and the landmarks predicted using EKF based on that trajectory. I selected the landmarks that appeared more than a hundred times, and there are over eight hundred in total. For part 4, when converting my landmark coordinates into the camera's pixel coordinates, there were significant errors; my EKF method works fine, but I couldn't achieve the desired results. Given an additional two hours at most, I could have gotten it running, but since the deadline has passed, I am unable to show the results—even though the logic of my code is correct.



(a) Dataset 01



(b) Dataset 02

Fig. 1: Estimated Trajectory and Landmarks via EKF